



## **Actividad Evaluativa 2 - Informe individual**

### **Módulo de Contacto y Consultas con JavaScript**

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación 3

COMISIÓN: "A"

PROFESOR: Sicardi Dante

ESTUDIANTE: Roa Santino

FECHA: 08-10-2026

REPOSITORIO: <https://github.com/RoaSantino/veterinaria-AE-1>



## INTRODUCCIÓN

Veterinaria Patitas es el sitio web de una veterinaria de Posadas, Misiones, desarrollado en la Actividad Evaluativa 1. En esa etapa construí el frontend de la tienda online con HTML y CSS: portada, listado de productos en tabla y en tarjetas, ficha de producto y formulario de compra. Esta Actividad Evaluativa 2 incorpora el módulo de Contacto y Consultas, que permite al cliente enviar una consulta sobre los servicios del local sin recargar la página.

Para resolverlo registré los eventos con `addEventListener`, validé los campos en el cliente, simulé el envío al servidor mediante `fetch` y `async/await` sobre un archivo JSON local y actualicé el DOM con el resultado. Para fundamentar el diseño apliqué el modelo EORM, que permite representar la consulta como un enlace enriquecido entre el cliente y el servicio.

En este informe presento el módulo desarrollado, su organización técnica, el escenario de uso, la fundamentación con EORM, el mapa navegacional, los estados de la interfaz y las verificaciones que realicé. El alcance es académico: la respuesta del servidor está simulada y las consultas no se almacenan de forma permanente.

### Alcance y trazabilidad

**Tabla 1**

*Requisitos de la AE2 y evidencia en la implementación*

| Requisito            | Evidencia en la implementación                                                                                                                                               |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Continuidad con AE1  | Se conservan las seis páginas, el menú común, los cinco productos y la hoja <code>style.css</code> .                                                                         |
| Eventos desacoplados | <code>submit</code> , <code>reset</code> e <code>input</code> se registran en <code>app.js</code> con <code>addEventListener</code> ; el HTML no tiene manejadores en línea. |
| HTTP y asincronía    | <code>fetch</code> y <code>async/await</code> consultan <code>respuesta.json</code> desde un servidor local; se verifica <code>response.ok</code> .                          |
| Validación y DOM     | Se aplica <code>.trim()</code> a los campos; los errores y el éxito se muestran en el DOM sin recargar.                                                                      |
| Metodología EORM     | Cliente y Servicio como clases; Consulta como enlace enriquecido con atributos propios.                                                                                      |



## DESARROLLO

### 1. Descripción del módulo

Incorpore el módulo en `contacto.html`, accesible desde el menú de navegación de todas las páginas. El formulario solicita nombre y apellido, e-mail, servicio (consulta general, vacunación, peluquería o cirugía) y el texto de la consulta. Junto al formulario agregué un panel de consultas enviadas, que muestra cada consulta registrada durante la visita con su servicio, cliente, fecha y estado.

Al enviar, el código valida los datos. Si falta algún campo o el e-mail no tiene un formato válido, marca los campos con error, informa cuáles hay que revisar y no realiza el envío. Si los datos son correctos, consulta `respuesta.json`, muestra el mensaje de confirmación que devuelve el archivo, registra la consulta en el panel y limpia el formulario.

### 2. Organización técnica

**Tabla 2**

*Archivos del módulo y su responsabilidad*

| Componente                  | Responsabilidad                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>contacto.html</code>  | Presenta el formulario, la zona de mensajes y el panel de consultas enviadas.                                 |
| <code>app.js</code>         | Registra eventos, valida, envía con <code>fetch</code> , crea los elementos del panel y gestiona los estados. |
| <code>respuesta.json</code> | Simula la respuesta del servidor con un estado y un mensaje de confirmación.                                  |
| <code>style.css</code>      | Mantiene el diseño de la AE1 y agrega estilos para mensajes, errores y panel de consultas.                    |

Implementé el envío en la función asíncrona `enviarConsulta()`. Primero llamo a `e.preventDefault()` para evitar la recarga, luego valido y, si todo es correcto, ejecuto `await fetch("respuesta.json")`. Verifico `response.ok` y que el JSON tenga el estado y el mensaje esperados antes de darlo por válido. Cualquier error HTTP, de red o de formato se lanza como excepción y lo captura el bloque `try/catch`, que muestra el problema al usuario [2].

Construyo los elementos del panel con `createElement` y `textContent` y los inserto con `replaceChildren`. De esta forma el texto que escribe el cliente se muestra siempre como texto y nunca se interpreta como HTML. Para generar los elementos recorro el



arreglo de consultas con Array.map [3].

### 3. Escenario de envío de una consulta

Actor principal: cliente de la veterinaria. Precondición: el sitio se sirve mediante HTTP (Live Server o GitHub Pages). Objetivo: enviar una consulta sobre un servicio y recibir una confirmación.

**Tabla 3**

*Flujo principal del escenario*

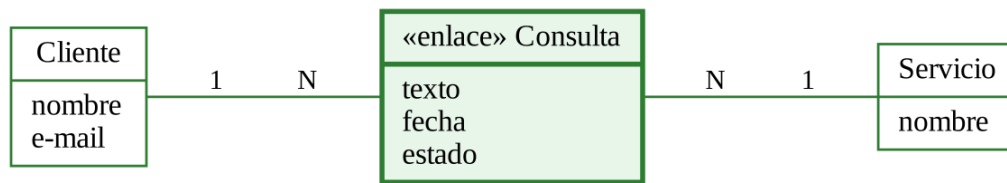
| Paso | Acción y respuesta esperada                                                                                 |
|------|-------------------------------------------------------------------------------------------------------------|
| 1    | Abrir Contacto desde el menú; el sistema muestra el formulario y el panel vacío.                            |
| 2    | Completar nombre, e-mail, elegir Vacunación y escribir la consulta.                                         |
| 3    | Presionar Enviar consulta; el botón cambia a Enviando y se bloquea.                                         |
| 4    | El sistema recibe la respuesta de respuesta.json y muestra el mensaje de éxito.                             |
| 5    | La consulta aparece en el panel con servicio, cliente, fecha y estado Recibida; el formulario queda limpio. |

Flujos alternativos: si algún campo queda vacío o el e-mail es inválido, se informa el error y se permite corregirlo sin perder lo escrito. Si falla la petición, se muestra un mensaje de error, el formulario conserva los datos y el usuario puede volver a enviar. La postcondición del flujo principal es una consulta registrada en la sesión; no se envía un correo real.

### 4. Fundamentación del modelo EORM

EORM (Enhanced Object Relationship Methodology) es una metodología de diseño hipermedia propuesta por Lange que parte del modelado orientado a objetos. Su idea central es tratar las relaciones entre objetos como clases con significado propio: los enlaces no son solo conexiones, sino objetos que pueden tener atributos y comportamiento. A esto se lo denomina enlace enriquecido [1].

En Veterinaria Patitas identifiqué dos clases principales: Cliente, con nombre y e-mail, y Servicio, con su nombre. Un cliente puede consultar por varios servicios y un servicio recibe consultas de muchos clientes. Esa relación, por sí sola, no guarda información. La Consulta es el enlace enriquecido entre ambos: además de unirlos, tiene datos propios, como el texto, la fecha de envío y el estado.



**Figura 1.** Modelo EORM: Consulta como enlace enriquecido entre Cliente y Servicio

**Tabla 4**

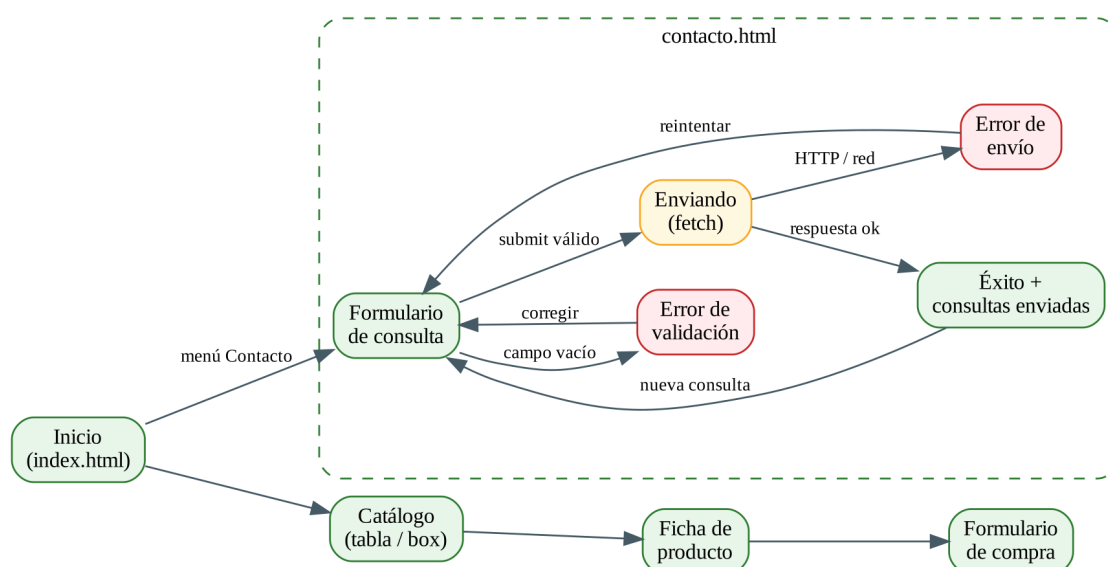
*Aplicación de EORM en el módulo*

| Elemento de EORM   | Aplicación en Veterinaria Patitas                                                        |
|--------------------|------------------------------------------------------------------------------------------|
| Clases de objetos  | Cliente (nombre, e-mail) y Servicio (consulta general, vacunación, peluquería, cirugía). |
| Enlace enriquecido | Consulta, con texto, fecha y estado, une a un cliente con un servicio.                   |
| Composición        | contacto.html agrupa el formulario que crea el enlace y el panel que lo muestra.         |
| Interfaz           | Cada campo del formulario corresponde a un atributo de Cliente, Servicio o Consulta.     |

En el código, cada consulta se guarda como un objeto que contiene al cliente, el servicio, el texto, la fecha y el estado. El panel de consultas enviadas hace visible ese enlace. Apliqué las ideas de EORM al diseño; no implementé una herramienta formal de la metodología.

## 5. Mapa navegacional

Derivé el mapa navegacional del escenario anterior. Conservé el recorrido de la AE1 (Inicio, catálogo, ficha y formulario de compra) y agregué el acceso a Contacto desde el menú. Representé la validación, el envío, el éxito y los errores como estados de contacto.html y no como páginas independientes.



**Figura 2.** Mapa navegacional del módulo de Contacto y Consultas

Desde cualquier estado de error el usuario vuelve al formulario sin perder lo que escribió. Después de un envío exitoso puede iniciar una nueva consulta. Guardé el diagrama editable en docs/mapa-navegacional-contacto.dot.

## 6. Interacción y estados de la interfaz

Vinculé el evento submit del formulario con `enviarConsulta()`, el evento reset con `limpiarFormulario()` y el evento input de cada campo con una función que quita la marca de error al corregirlo. Registré todos los eventos con `addEventListener` en `app.js` y no usé atributos `onclick` ni `onsubmit` en el HTML [4].

Durante el envío deshabilito el botón y cambio su texto a `Enviando` para evitar envíos duplicados; al terminar, con éxito o con error, el bloque `finally` lo restablece. La zona de mensajes usa `role="status"` y `aria-live="polite"` para que los lectores de pantalla anuncien el resultado, y el formulario indica la carga con `aria-busy`. En teléfonos, el formulario y el panel se ubican en una sola columna.

## 7. Verificación

Ejecuté 21 comprobaciones automatizadas en Chromium con un servidor HTTP local. Comprobé la validación de campos vacíos y con solo espacios, el formato del e-mail, el envío correcto, el registro de consultas en el panel, la limpieza del formulario, los errores HTTP y de red, la inserción de texto como texto, la vista móvil y la carga del resto de las páginas de la AE1.



**Tabla 5**

*Casos representativos*

| Caso                           | Resultado observado                                                                                 |
|--------------------------------|-----------------------------------------------------------------------------------------------------|
| Formulario vacío               | Se marcan los cuatro campos y se informa qué revisar; la página no se recarga.                      |
| Nombre y consulta con espacios | .trim() los detecta como vacíos y no se envía nada.                                                 |
| Envío correcto                 | Se muestra el mensaje de respuesta.json, la consulta aparece en el panel y el formulario se limpia. |
| Consulta con etiquetas HTML    | El texto se muestra literalmente; no se crean elementos HTML.                                       |
| Error HTTP 404 o falla de red  | try/catch muestra el error, se conservan los datos y el botón se rehabilita.                        |
| Vista de 390 píxeles           | No existe desbordamiento horizontal.                                                                |

Obtuve 21 verificaciones correctas y ningún error de JavaScript en el recorrido. Incluí capturas de pantalla en los anexos.

## 8. Decisiones técnicas

Decidí validar en el cliente antes de hacer la petición, para no enviar datos incompletos y dar una respuesta inmediata. Usé el atributo novalidate en el formulario para que la validación la resuelva mi código y los mensajes sean siempre los mismos en cualquier navegador.

Simulé el servidor con una petición GET a respuesta.json, porque un archivo estático no acepta peticiones POST en Live Server ni en GitHub Pages. Aun así traté la respuesta como si viniera de un servidor real: verifico el código HTTP y el formato de los datos. Mantuve el código simple, con funciones cortas y nombres descriptivos, para poder explicar cada paso.



## CONCLUSIONES FINALES

En esta Actividad Evaluativa 2 incorporé a Veterinaria Patitas el módulo de Contacto y Consultas, complementario al catálogo y al formulario de compra de la AE1. Cumplí los requisitos al registrar eventos desacoplados con `addEventListener`, validar los datos con `.trim()`, simular el envío de forma asíncrona con `fetch` y `async/await` y actualizar el DOM con mensajes de éxito o error.

El modelo EORM me ayudó a pensar la consulta como algo más que un formulario: es el enlace que une a un cliente con un servicio y que guarda información propia. A partir del escenario definí los campos, los estados de la interfaz y el mapa navegacional, y con las pruebas comprobé tanto el caso normal como las rutas de error.

Como mejora futura, propongo reemplazar `respuesta.json` por un servidor real que reciba las consultas con POST, las almacene y permita a la veterinaria cambiar su estado a Respondida.

## BIBLIOGRAFÍA

- [1] Lange, D. B. (1994). An object-oriented design method for hypermedia information systems. *Proceedings of the 27th Hawaii International Conference on System Sciences*. IEEE.
- [2] MDN Web Docs. (s. f.). Using the Fetch API.  
[https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)
- [3] MDN Web Docs. (s. f.). `Array.prototype.map()`. [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Array/map](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map)
- [4] MDN Web Docs. (s. f.). `EventTarget.addEventListener()`.  
<https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener>
- [5] Roa, S. (2026). *Startup Relámpago: Veterinaria Patitas* [Actividad Evaluativa 1]. Repositorio del proyecto. <https://github.com/RoaSantino/veterinaria-AE-1>



## ANEXOS

### Anexo A. Envío correcto

**Veterinaria Patitas** Inicio Productos (tabla) Productos (box) Comprar Contacto

### Contacto y consultas

Escribinos tu consulta sobre cualquiera de nuestros servicios y te respondemos por e-mail.

**Nombre y apellido**  
Ej: Juan Pérez

**E-mail**  
ejemplo@correo.com

**Servicio**  
Elegí un servicio

**Consulta**  
Contanos qué necesitás

¡Gracias! Recibimos tu consulta y te vamos a responder por e-mail dentro de las 24 horas.

Enviar consulta Limpiar

#### Consultas enviadas

**Vacunación** Recibida  
Ana Gómez - 8/10/26, 10:57 a. m.  
¿Qué vacunas necesita un cachorro de 2 meses?

**Peluquería** Recibida  
Luis Benítez - 8/10/26, 10:57 a. m.  
¿Atienden sábados?

Veterinaria Patitas - Posadas, Misiones - 2026

*Figura 3. Mensaje de éxito y consultas registradas en el panel*

### Anexo B. Validación de campos

**Veterinaria Patitas** Inicio Productos (tabla) Productos (box) Comprar Contacto

### Contacto y consultas

Escribinos tu consulta sobre cualquiera de nuestros servicios y te respondemos por e-mail.

**Nombre y apellido**  
Ej: Juan Pérez

**E-mail**  
ejemplo@correo.com

**Servicio**  
Elegí un servicio

**Consulta**  
Contanos qué necesitás

Revisá los siguientes datos: nombre, e-mail, servicio, consulta.

Enviar consulta Limpiar

#### Consultas enviadas

Todavía no enviaste consultas en esta visita.

Veterinaria Patitas - Posadas, Misiones - 2026

*Figura 4. Formulario vacío: campos marcados y mensaje de error*



## Anexo C. Error en la petición

The screenshot shows the 'Contacto y consultas' page of the 'Veterinaria Patitas' website. The page has a green header with navigation links: Inicio, Productos (tabla), Productos (box), Comprar, and Contacto. The main content area is titled 'Contacto y consultas' and includes a sub-header 'Escribimos tu consulta sobre cualquiera de nuestros servicios y te respondemos por e-mail.' Below this is a form with fields for 'Nombre y apellido' (filled with 'Ana'), 'E-mail' (filled with 'ana@mail.com'), 'Servicio' (a dropdown menu with 'Vacunación' selected), and 'Consulta' (filled with 'Hola'). At the bottom of the form are two buttons: 'Enviar consulta' and 'Limpiar'. A red error message box is displayed below the form, stating: 'No pudimos enviar tu consulta (el servidor respondió con el código 404). Intentá nuevamente.' To the right of the form is a box titled 'Consultas enviadas' with the text 'Todavía no enviaste consultas en esta visita.' The footer of the page reads 'Veterinaria Patitas - Posadas, Misiones - 2026'.

*Figura 5. Respuesta HTTP 404: mensaje de error con los datos conservados*

## Anexo D. Estructura de archivos del repositorio

- index.html
- listado\_tabla.html
- listado\_box.html
- producto.html
- comprar.html
- contacto.html
- style.css
- app.js
- respuesta.json
- docs/mapa-navegacional-contacto.dot
- docs/modelo-eorm.dot